

Kotlin Generics

From Fundamentals to Advanced

A Comprehensive Guide with 50+ Examples

Table of Contents

1. Introduction to Generics
2. Level 1: Basic Generics (Examples 1-10)
3. Level 2: Generic Constraints (Examples 11-20)
4. Level 3: Variance (Examples 21-30)
5. Level 4: Advanced Patterns (Examples 31-40)
6. Level 5: Complex Scenarios (Examples 41-50)
7. Practice Questions

1. Introduction to Generics

Generics allow you to write flexible, reusable code by parameterizing types. Instead of writing separate classes or functions for each type, you can write one that works with any type while maintaining type safety.

Why Use Generics?

- Type safety: Catch errors at compile time instead of runtime
- Code reusability: Write once, use with multiple types
- Eliminate casting: No need for explicit type conversions
- Better IDE support: Auto-completion and type inference

2. Level 1: Basic Generics

Example 1: Generic Class

```
class Box<T>(val item: T) { fun get(): T = item } val intBox = Box(42) val stringBox = Box("Hello") println(intBox.get()) // 42 println(stringBox.get()) // Hello
```

Explanation: A simple generic class that can hold any type. The type parameter T is specified in angle brackets.

Example 2: Generic Function

```
fun <T> identity(value: T): T { return value } val num = identity(10) // T inferred as Int val text = identity("Kotlin") // T inferred as String
```

Explanation: Generic functions allow type parameters to be inferred from the arguments passed.

Example 3: Multiple Type Parameters

```
class Pair<A, B>(val first: A, val second: B) { fun printTypes() { println("First: ${first}, Second: ${second}") } } val pair = Pair(1, "One") pair.printTypes() // First: 1, Second: One
```

Explanation: You can use multiple type parameters separated by commas.

Example 4: Generic List Processing

```
fun <T> printList(items: List<T>) { for (item in items) { println(item) } } printList(listOf(1, 2, 3)) printList(listOf("a", "b", "c"))
```

Explanation: Generic functions work seamlessly with Kotlin's collection types.

Example 5: Generic Return Type

```
fun <T> getFirst(list: List<T>): T? { return list.firstOrNull() } val first: Int? = getFirst(listOf(1, 2, 3)) val firstStr: String? = getFirst(listOf("a", "b"))
```

Explanation: Functions can return generic types, with the actual type determined by usage.

Example 6: Generic Interface

```
interface Container<T> { fun store(item: T) fun retrieve(): T? } class
SimpleContainer<T> : Container<T> { private var item: T? = null override fun
store(item: T) { this.item = item } override fun retrieve(): T? = item }
```

Explanation: Interfaces can be generic, and implementing classes must specify or propagate the type parameter.

Example 7: Extension Function with Generics

```
fun <T> List<T>.secondOrNull(): T? { return if (size >= 2) this[1] else null } val
list = listOf(1, 2, 3) println(list.secondOrNull()) // 2
```

Explanation: Extension functions can be generic, adding functionality to any type.

Example 8: Generic Data Class

```
data class Result<T> ( val data: T?, val error: String? ) val success = Result(data =
42, error = null) val failure = Result<Int>(data = null, error = "Failed")
```

Explanation: Data classes work perfectly with generics for creating type-safe result or response wrappers.

Example 9: Nullable Generic Types

```
class Wrapper<T>(val value: T) { fun isNull(): Boolean = value == null } val
wrapper1 = Wrapper<String?>(null) val wrapper2 = Wrapper("Hello")
println(wrapper1.isNull()) // true println(wrapper2.isNull()) // false
```

Explanation: Generic type parameters can be nullable or non-nullable.

Example 10: Generic with Default Value

```
class ValueHolder<T>(private val defaultValue: T) { private var value: T? = null fun
get(): T = value ?: defaultValue fun set(newValue: T) { value = newValue } } val
holder = ValueHolder(0) println(holder.get()) // 0 holder.set(42)
println(holder.get()) // 42
```

Explanation: Generic classes can have default values of the generic type.

3. Level 2: Generic Constraints

Generic constraints restrict the types that can be used as type arguments. This enables you to call specific methods or access properties that the constraint guarantees.

Example 11: Upper Bound Constraint

```
fun <T : Number> sum(a: T, b: T): Double { return a.toDouble() + b.toDouble() } val  
result1 = sum(5, 10) // Int val result2 = sum(3.14, 2.86) // Double // val result3 =  
sum("1", "2") // Error: String is not a subtype of Number
```

Explanation: Upper bound constraints limit type parameters to a specific type or its subtypes.

Example 12: Comparable Constraint

```
fun <T : Comparable<T>> findMax(a: T, b: T): T { return if (a > b) a else b }  
println(findMax(10, 20)) // 20 println(findMax("apple", "banana")) // banana
```

Explanation: Constraints enable us to use specific operations like comparison.

Example 13: Multiple Constraints (Where Clause)

```
interface Named { val name: String } fun <T> printInfo(item: T) where T :  
Comparable<T>, T : Named { println("Name: ${item.name}") } data class  
Person(override val name: String, val age: Int) : Named, Comparable<Person> {  
    override fun compareTo(other: Person) = age - other.age } printInfo(Person("Alice",  
30))
```

Explanation: The where clause allows multiple constraints on a single type parameter.

Example 14: Generic Class with Constraint

```
class NumberBox<T : Number>(val value: T) { fun asDouble(): Double =  
value.toDouble() fun asInt(): Int = value.toInt() } val intBox = NumberBox(42) val  
doubleBox = NumberBox(3.14) println(intBox.asDouble()) // 42.0
```

Explanation: Constraints in class declarations ensure all instances work with the constrained type.

Example 15: Reified Type Parameters

```
inline fun <reified T> isInstance(value: Any): Boolean { return value is T }  
println(isInstance<String>("Hello")) // true println(isInstance<Int>("Hello")) //  
false
```

Explanation: Reified type parameters allow you to access the actual type at runtime in inline functions.

Example 16: Filtering with Reified Types

```
inline fun <reified T> List<*>.filterIsInstance(): List<T> { return this.filter { it is T }.map { it as T } } val mixed = listOf(1, "two", 3, "four", 5) val numbers = mixed.filterIsInstance<Int>() println(numbers) // [1, 3, 5]
```

Explanation: Reified types are powerful for type-safe filtering and conversion operations.

Example 17: Star Projection

```
fun printSize(list: List<*>) { println("Size: ${list.size}") // Can't access elements with specific type } printSize(listOf(1, 2, 3)) printSize(listOf("a", "b"))
```

Explanation: Star projection (*) is used when you don't know or care about the specific type parameter.

Example 18: Generic Constraint with Custom Interface

```
interface Identifiable { val id: String } class Repository<T : Identifiable> { private val items = mutableListOf<T>() fun add(item: T) { items.add(item) } fun findById(id: String): T? { return items.find { it.id == id } } } data class User(override val id: String, val name: String) : Identifiable
```

Explanation: Constraints with custom interfaces enable type-safe repositories and data access patterns.

Example 19: Recursive Type Bounds

```
interface SelfComparable<T : SelfComparable<T>> : Comparable<T> class Version(val major: Int, val minor: Int) : SelfComparable<Version> { override fun compareTo(other: Version): Int { return when { major != other.major -> major - other.major else -> minor - other.minor } } }
```

Explanation: Recursive type bounds ensure a type can only be compared with itself, not other types.

Example 20: Generic Builder Pattern

```
class QueryBuilder<T : Any> { private val conditions = mutableListOf<String>() fun where(condition: String): QueryBuilder<T> { conditions.add(condition) return this } fun build(): String { return "SELECT * WHERE " + conditions.joinToString(" AND ") } } val query = QueryBuilder<User>().where("age > 18").where("active = true").build()
```

Explanation: Generic builders create fluent APIs while maintaining type safety.

4. Level 3: Variance (Covariance and Contravariance)

Variance describes how subtyping between complex types relates to subtyping between their component types. Kotlin supports declaration-site variance (out/in) and use-site variance (type projections).

Example 21: Covariance with 'out'

```
interface Producer<out T> { fun produce(): T } class StringProducer :  
    Producer<String> { override fun produce() = "Hello" } val stringProducer:  
    Producer<String> = StringProducer() val anyProducer: Producer<Any> = stringProducer  
    // OK with 'out'
```

Explanation: The 'out' modifier makes a type parameter covariant, meaning it can only be produced (returned), not consumed.

Example 22: Contravariance with 'in'

```
interface Consumer<in T> { fun consume(item: T) } class AnyConsumer : Consumer<Any>  
{ override fun consume(item: Any) { println("Consuming: $item") } } val anyConsumer:  
    Consumer<Any> = AnyConsumer() val stringConsumer: Consumer<String> = anyConsumer //  
    OK with 'in'
```

Explanation: The 'in' modifier makes a type parameter contravariant, meaning it can only be consumed (passed in), not produced.

Example 23: Invariance (No Modifier)

```
class Box<T>(var item: T) { fun get(): T = item fun set(value: T) { item = value } }  
val stringBox = Box("Hello") // val anyBox: Box<Any> = stringBox // Error: invariant
```

Explanation: Without variance modifiers, a type parameter is invariant - it can be both produced and consumed but no subtyping is allowed.

Example 24: List Covariance

```
val strings: List<String> = listOf("a", "b", "c") val anys: List<Any> = strings //  
    OK - List is covariant // This works because List<out T> is read-only //  
    MutableList<String> cannot be assigned to MutableList<Any>
```

Explanation: Kotlin's List is covariant (List<out T>) because it's read-only and only produces elements.

Example 25: Use-Site Variance (Type Projection)

```
fun copy(from: Array<out Any>, to: Array<Any>) { for (i in from.indices) { to[i] =  
    from[i] } } val ints = arrayOf(1, 2, 3) val anys = arrayOf<Any>("a", "b", "c")  
    copy(ints, anys) // ints projected as Array<out Any>
```

Explanation: Use-site variance allows you to project a type with variance at the point of use.

Example 26: Contravariant Type Projection

```
fun fill(dest: Array<in String>, value: String) { for (i in dest.indices) { dest[i] = value } } val strings = arrayOfNulls<String>(3) val anys = arrayOfNulls<Any>(3) fill(strings, "test") fill(anys, "test") // Array<Any> treated as Array<in String>
```

Explanation: The 'in' projection allows you to write to an array of a supertype.

Example 27: Generic Function with Out Projection

```
fun printAll(items: List<out Any>) { for (item in items) { println(item) } } printAll(listOf(1, 2, 3)) printAll(listOf("a", "b", "c"))
```

Explanation: Out projections in function parameters allow accepting subtypes safely.

Example 28: Covariant Return Types

```
open class Animal { open fun makeSound(): String = "Some sound" } class Dog : Animal() { override fun makeSound(): String = "Bark" } interface AnimalFactory<out T : Animal> { fun create(): T } class DogFactory : AnimalFactory<Dog> { override fun create() = Dog() }
```

Explanation: Covariance is natural for factory patterns where you only produce values.

Example 29: Contravariant Comparator

```
interface Comparator<in T> { fun compare(a: T, b: T): Int } class AnyComparator : Comparator<Any> { override fun compare(a: Any, b: Any): Int { return a.hashCode() - b.hashCode() } } val stringComparator: Comparator<String> = AnyComparator()
```

Explanation: Comparators are naturally contravariant because they consume values to compare them.

Example 30: Variance in Higher-Order Functions

```
fun <T> process( items: List<T>, producer: () -> T, consumer: (T) -> Unit ) { consumer(producer()) items.forEach(consumer) } process( listOf(1, 2, 3), producer = { 0 }, consumer = { println(it) } )
```

Explanation: Function types have built-in variance: contravariant in parameters, covariant in return types.

5. Level 4: Advanced Generic Patterns

Example 31: Generic Type Alias

```
typealias StringMap<T> = Map<String, T> typealias Predicate<T> = (T) -> Boolean val
scores: StringMap<Int> = mapOf("Alice" to 95, "Bob" to 87) val isEven:
Predicate<Int> = { it % 2 == 0 }
```

Explanation: Type aliases simplify complex generic type declarations.

Example 32: Phantom Types

```
sealed class State object Draft : State() object Published : State() class
Document<S : State> private constructor(val content: String) { companion object {
fun create(content: String): Document<Draft> { return Document(content) } } fun
publish(): Document<Published> { return Document(content) } } val doc =
Document.create("Hello") val published = doc.publish() // val repub =
published.publish() // Error: no publish() on Published
```

Explanation: Phantom types encode state in the type system, preventing invalid operations at compile time.

Example 33: Generic Delegation

```
interface Repository<T> { fun save(item: T) fun findAll(): List<T> } class
CachingRepository<T>( private val delegate: Repository<T> ) : Repository<T> by
delegate { private val cache = mutableListOf<T>() override fun save(item: T) {
delegate.save(item) cache.add(item) } }
```

Explanation: Generic delegation patterns allow wrapping behavior while maintaining type safety.

Example 34: Self-Referential Generics

```
abstract class Builder<T : Builder<T>> { abstract fun self(): T fun withLogging(): T
{ println("Logging enabled") return self() } } class QueryBuilder :
Builder<QueryBuilder>() { private var table: String = "" override fun self() = this
fun from(table: String): QueryBuilder { this.table = table return this } }
```

Explanation: Self-referential generics enable the curiously recurring template pattern for fluent APIs.

Example 35: Generic Sealed Classes

```
sealed class Result<out T> { data class Success<T>(val value: T) : Result<T>() data
class Error(val message: String) : Result<Nothing>() object Loading :
Result<Nothing>() } fun <T> handleResult(result: Result<T>) { when (result) { is
Result.Success -> println("Got: ${result.value}") is Result.Error -> println("Error:
${result.message}") Result.Loading -> println("Loading...") } }
```

Explanation: Generic sealed classes are perfect for modeling state and results with type safety.

Example 36: Higher-Kinded Types Simulation

```
interface Functor<F, A> { fun <B> map(f: (A) -> B): Functor<F, B> } class
ListFunctor<A>(private val items: List<A>) : Functor<List<*>, A> { override fun <B>
map(f: (A) -> B): Functor<List<*>, B> { return ListFunctor(items.map(f)) } }
```

Explanation: While Kotlin doesn't have true higher-kinded types, we can simulate them with careful design.

Example 37: Context Receivers Pattern (Simulated)

```
interface Logger { fun log(message: String) } class LoggingContext : Logger {
override fun log(message: String) { println("[LOG] $message") } } fun <T>
withLogging(context: Logger, block: Logger.() -> T): T { return context.block() }
withLogging(LoggingContext()) { log("Operation started") log("Operation completed")
}
```

Explanation: Generic context patterns enable dependency injection and scoped behavior.

Example 38: Type-Safe Builder DSL

```
class HTML { private val children = mutableListOf<String>() fun body(init: Body.()
-> Unit) { children.add(Body().apply(init).toString()) } override fun toString() =
"<html>${children.joinToString("")}</html>" } class Body { private val children =
mutableListOf<String>() fun p(text: String) { children.add("<p>$text</p>") }
override fun toString() = "<body>${children.joinToString("")}</body>" } fun
html(init: HTML.() -> Unit): HTML = HTML().apply(init) val page = html { body {
p("Hello, World!") } }
```

Explanation: Generic extension functions enable type-safe DSLs for domain-specific languages.

Example 39: Generic Event System

```
interface Event data class UserLoggedIn(val userId: String) : Event data class
DataUpdated<T>(val data: T) : Event class EventBus { private val handlers =
mutableMapOf<Class<*>, MutableList<Event> -> Unit>() inline fun <reified T :
Event> subscribe(noinline handler: (T) -> Unit) { handlers.getOrPut(T::class.java) {
mutableListOf() } .add { handler(it as T) } } fun publish(event: Event) {
handlers[event::class.java]?.forEach { it(event) } }
```

Explanation: Reified generics enable type-safe event systems without explicit class tokens.

Example 40: Monad-like Pattern

```
class Optional<out T> private constructor(private val value: T?) { companion object
{ fun <T> of(value: T?): Optional<T> = Optional(value) fun <T> empty(): Optional<T>
= Optional(null) } fun <R> map(transform: (T) -> R): Optional<R> { return if (value
!= null) of(transform(value)) else empty() } fun <R> flatMap(transform: (T) ->
Optional<R>): Optional<R> { return if (value != null) transform(value) else empty()
} fun getOrElse(default: T): T = value ?: default } val result = Optional.of("42")
.map { it.toInt() } .map { it * 2 } .getOrElse(0)
```

Explanation: Monadic patterns with generics enable functional composition and safe null handling.

6. Level 5: Complex Real-World Scenarios

Example 41: Generic Repository with Specification Pattern

```
interface Specification<T> { fun isSatisfiedBy(item: T): Boolean } class
AndSpecification<T>( private val specs: List<Specification<T>> ) : Specification<T>
{ override fun isSatisfiedBy(item: T) = specs.all { it.isSatisfiedBy(item) } } class
GenericRepository<T> { private val items = mutableListOf<T>() fun add(item: T) =
items.add(item) fun find(spec: Specification<T>): List<T> { return items.filter {
spec.isSatisfiedBy(it) } } }
```

Explanation: Combining generics with design patterns creates flexible, testable architectures.

Example 42: Generic Dependency Injection Container

```
class DIContainer { private val factories = mutableMapOf<Class<*>, () -> Any>()
inline fun <reified T : Any> register(noinline factory: () -> T) {
factories[T::class.java] = factory } inline fun <reified T : Any> resolve(): T { val
factory = factories[T::class.java] ?: error("No factory registered for
${T::class.java}") return factory() as T } } interface Service class ServiceImpl :
Service val container = DIContainer() container.register<Service> { ServiceImpl() }
val service = container.resolve<Service>()
```

Explanation: Reified generics enable compile-time type-safe dependency injection.

Example 43: Generic State Machine

```
interface State<T> { fun handle(event: T): State<T> } class
StateMachine<T>(initialState: State<T>) { private var currentState = initialState
fun send(event: T) { currentState = currentState.handle(event) } fun
getCurrentState() = currentState } sealed class TrafficLight : State<Unit> { object
Red : TrafficLight() { override fun handle(event: Unit) = Green } object Green :
TrafficLight() { override fun handle(event: Unit) = Yellow } object Yellow :
TrafficLight() { override fun handle(event: Unit) = Red } }
```

Explanation: Generic state machines provide type-safe event handling and state transitions.

Example 44: Parser Combinator

```
interface Parser<out T> { fun parse(input: String): Pair<T?, String> } class
LiteralParser(private val literal: String) : Parser<String> { override fun
parse(input: String): Pair<String?, String> { return if (input.startsWith(literal))
{ literal to input.substring(literal.length) } else { null to input } } } fun <A, B>
Parser<A>.andThen(other: Parser<B>): Parser<Pair<A, B>> { return object :
Parser<Pair<A, B>> { override fun parse(input: String): Pair<Pair<A, B>?, String> {
val (a, rest1) = this@andThen.parse(input) if (a == null) return null to input val
(b, rest2) = other.parse(rest1) return if (b != null) (a to b) to rest2 else null to
input } } }
```

Explanation: Generics enable composable parser combinators for building complex parsers.

Example 45: Type-Safe Query Builder

```
sealed class Column<T>(val name: String) class IntColumn(name: String) :
Column<Int>(name) class StringColumn(name: String) : Column<String>(name) class
Table(val name: String) { val id = IntColumn("id") val email = StringColumn("email") }
```

```
} class Query<T>(private val column: Column<T>, private val value: T) { fun toSql()  
= "SELECT * FROM table WHERE ${column.name} = '$value'" } val users = Table("users")  
val query = Query(users.email, "test@example.com")
```

Explanation: Generic columns ensure type-safe database query construction.

Example 46: Lens Pattern for Immutable Updates

```
class Lens<S, A> { val get: (S) -> A, val set: (S, A) -> S } { fun modify(s: S, f: (A) -> A): S = set(s, f(get(s))) fun <B> compose(other: Lens<A, B>): Lens<S, B> { return Lens( get = { s -> other.get(this.get(s)) }, set = { s, b -> this.set(s, other.set(this.get(s), b)) } ) } } data class Address(val street: String, val city: String) data class Person(val name: String, val address: Address) val addressLens = Lens<Person, Address>( get = { it.address }, set = { p, a -> p.copy(address = a) } ) val cityLens = Lens<Address, String>( get = { it.city }, set = { a, c -> a.copy(city = c) } ) val personCityLens = addressLens.compose(cityLens)
```

Explanation: Lenses provide functional updates to nested immutable data structures with composability.

Example 47: Generic Caching with TTL

```
data class CacheEntry<T>(val value: T, val expiresAt: Long) class Cache<K, V>(private val ttlMillis: Long) { private val cache = mutableMapOf<K, CacheEntry<V>>() fun put(key: K, value: V) { val expiresAt = System.currentTimeMillis() + ttlMillis cache[key] = CacheEntry(value, expiresAt) } fun get(key: K): V? { val entry = cache[key] ?: return null return if (System.currentTimeMillis() < entry.expiresAt) { entry.value } else { cache.remove(key) null } } }
```

Explanation: Generic caching systems handle any key-value types with expiration logic.

Example 48: Visitor Pattern with Generics

```
interface Element { fun <T> accept(visitor: Visitor<T>): T } interface Visitor<T> { fun visitInt(value: Int): T fun visitString(value: String): T } class IntElement(val value: Int) : Element { override fun <T> accept(visitor: Visitor<T>) = visitor.visitInt(value) } class StringElement(val value: String) : Element { override fun <T> accept(visitor: Visitor<T>) = visitor.visitString(value) } class StringifyVisitor : Visitor<String> { override fun visitInt(value: Int) = "Int: $value" override fun visitString(value: String) = "String: $value" }
```

Explanation: Generic visitors enable type-safe operations on heterogeneous structures.

Example 49: Asynchronous Result Handling

```
sealed class AsyncResult<out T> { object Loading : AsyncResult<Nothing>() data class Success<T>(val data: T) : AsyncResult<T>() data class Failure(val error: Throwable) : AsyncResult<Nothing>() fun <R> map(transform: (T) -> R): AsyncResult<R> { return when (this) { is Success -> Success(transform(data)) is Failure -> this Loading -> Loading } } suspend fun <R> flatMap( transform: suspend (T) -> AsyncResult<R> ): AsyncResult<R> { return when (this) { is Success -> transform(data) is Failure -> this Loading -> Loading } } }
```

Explanation: Generic async results handle loading states and errors uniformly across different data types.

Example 50: Type-Safe Configuration DSL

```
class ConfigBuilder<T> { private val properties = mutableMapOf<String, Any>() fun <V> property(name: String, value: V) { properties[name] = value as Any } inline fun <reified V> get(name: String): V? { return properties[name] as? V } } class AppConfig { var port: Int = 8080 var host: String = "localhost" var debug: Boolean =
```

```
false } fun config(init: ConfigBuilder<AppConfig>().-> Unit):  
ConfigBuilder<AppConfig> { return ConfigBuilder<AppConfig>().apply(init) } val  
appConfig = config { property("port", 9000) property("host", "0.0.0.0")  
property("debug", true) }
```

Explanation: Generic DSL builders create type-safe configuration with compile-time validation.

7. Practice Questions

Test your understanding of Kotlin generics with these questions. Try to answer them before checking the solutions.

Question 1: What is the difference between `List<String>` and `MutableList<String>` in terms of variance?

Answer: `List<out T>` is covariant because it's read-only, while `MutableList<T>` is invariant because it supports both reading and writing.

Question 2: Why can't you use 'out' variance on a type parameter that appears in a function parameter position?

Answer: Because 'out' means the type can only be produced (returned), not consumed (accepted as parameter). Allowing it in parameter position would break type safety.

Question 3: What does the 'reified' keyword do and when can it be used?

Answer: Reified preserves type information at runtime, allowing you to check types with 'is' operator. It can only be used with inline functions.

Question 4: Explain the difference between `Array<Int>` and `IntArray`.

Answer: `Array<Int>` is a boxed array (objects), while `IntArray` is a primitive array. `IntArray` is more memory efficient but `Array<Int>` is a generic type.

Question 5: What is a star projection (*) and when would you use it?

Answer: Star projection represents an unknown type. Use it when you don't know or care about the specific type parameter, like `List<*>` for any list.

Question 6: Can you explain why 'in' variance is called contravariance?

Answer: It's contravariant because the subtyping relationship is reversed: if `Dog <: Animal`, then `Consumer<Animal> <: Consumer<Dog>`.

Question 7: What's the difference between `<T : Any>` and `<T>` as type constraints?

Answer: `<T : Any>` means T cannot be nullable, while `<T>` allows nullable types (T includes T? implicitly).

Question 8: How do you create a generic function that works only with Number types?

Answer: Use an upper bound constraint: `fun <T : Number> myFunction(value: T) { ... }`

Question 9: What is declaration-site variance vs use-site variance?

Answer: Declaration-site: variance specified in class/interface definition (class `Producer<out T>`).
Use-site: variance specified when using the type (`Array<out Any>`).

Question 10: Why does Kotlin have both 'out' and 'in' variance modifiers?

Answer: 'Out' for types that only produce values (covariance), 'in' for types that only consume values (contravariance). Together they cover all safe variance scenarios.

Question 11: Can you have multiple type parameters with different variance modifiers?

Answer: Yes! For example: interface `Function<in T, out R>` represents a function from T to R, contravariant in parameter, covariant in return.

Question 12: What happens if you try to use a contravariant type parameter in a return position?

Answer: Compile error. Contravariant (in) type parameters can only be consumed (used in parameter positions), not produced (returned).

Question 13: Explain the 'where' clause in generic constraints.

Answer: The 'where' clause allows multiple constraints on type parameters: `fun <T> myFun(x: T) where T : Comparable<T>, T : Serializable`

Question 14: What is type erasure and how does it affect generics in Kotlin?

Answer: Type erasure means generic type information is removed at runtime. You can't check 'is `List<String>`' at runtime. Use reified in inline functions to preserve types.

Question 15: Can you create an array of a generic type T?

Answer: Not directly (no `Array<T>(size)`). Use `Array(size) { ... }` with a lambda, or `arrayOfNulls<T>(size)` for nullable arrays, or use reified type with inline functions.

Question 16: What's the purpose of the `Nothing` type in Kotlin's type hierarchy?

Answer: `Nothing` is the bottom type (subtype of everything). Used for functions that never return (throw exceptions) and as a type argument when no value is possible.

Question 17: How do recursive type bounds work? Give an example.

Answer: A type parameter can reference itself in its bound: `interface Comparable<T : Comparable<T>>`. This ensures a type can only compare with itself.

Question 18: What's the difference between `<T>` and `<T : Any?>` in terms of nullability?

Answer: `<T>` allows nullable types (T can be `String?`). `<T : Any>` restricts T to non-nullable types only.

Question 19: Can you override a function with different generic type parameters?

Answer: No, you can't change the number or constraints of type parameters in overrides. The signature must match including type parameters.

Question 20: What are phantom types and what problem do they solve?

Answer: Phantom types are type parameters that don't appear in the actual data but encode state in the type system, preventing invalid operations at compile time (e.g., `Document<Draft>` vs `Document<Published>`).

Coding Challenges

Try implementing these yourself to solidify your understanding:

1. Create a generic `Stack<T>` class with push, pop, and peek operations.
2. Implement a generic binary tree with in-order traversal.
3. Build a type-safe event bus that can handle any event type.
4. Create a generic `Result<T, E>` type for error handling similar to Rust.
5. Implement a generic LRU cache with configurable size.
6. Build a type-safe builder pattern for creating complex objects.
7. Create a generic validation framework with composable validators.
8. Implement a simple expression evaluator using generics.
9. Build a generic state machine with compile-time state transitions.
10. Create a generic observer pattern implementation.

Summary

Kotlin generics provide powerful abstractions for writing type-safe, reusable code. Key concepts include type parameters, constraints (upper bounds, where clause), variance (covariance with 'out', contravariance with 'in'), reified types for runtime type information, and advanced patterns like phantom types and type-safe builders. Practice these concepts regularly to master generic programming in Kotlin.